

Seals MES System 异地分布式部署方案

一、现状分析与分布式化瓶颈

1.1 当前架构局限

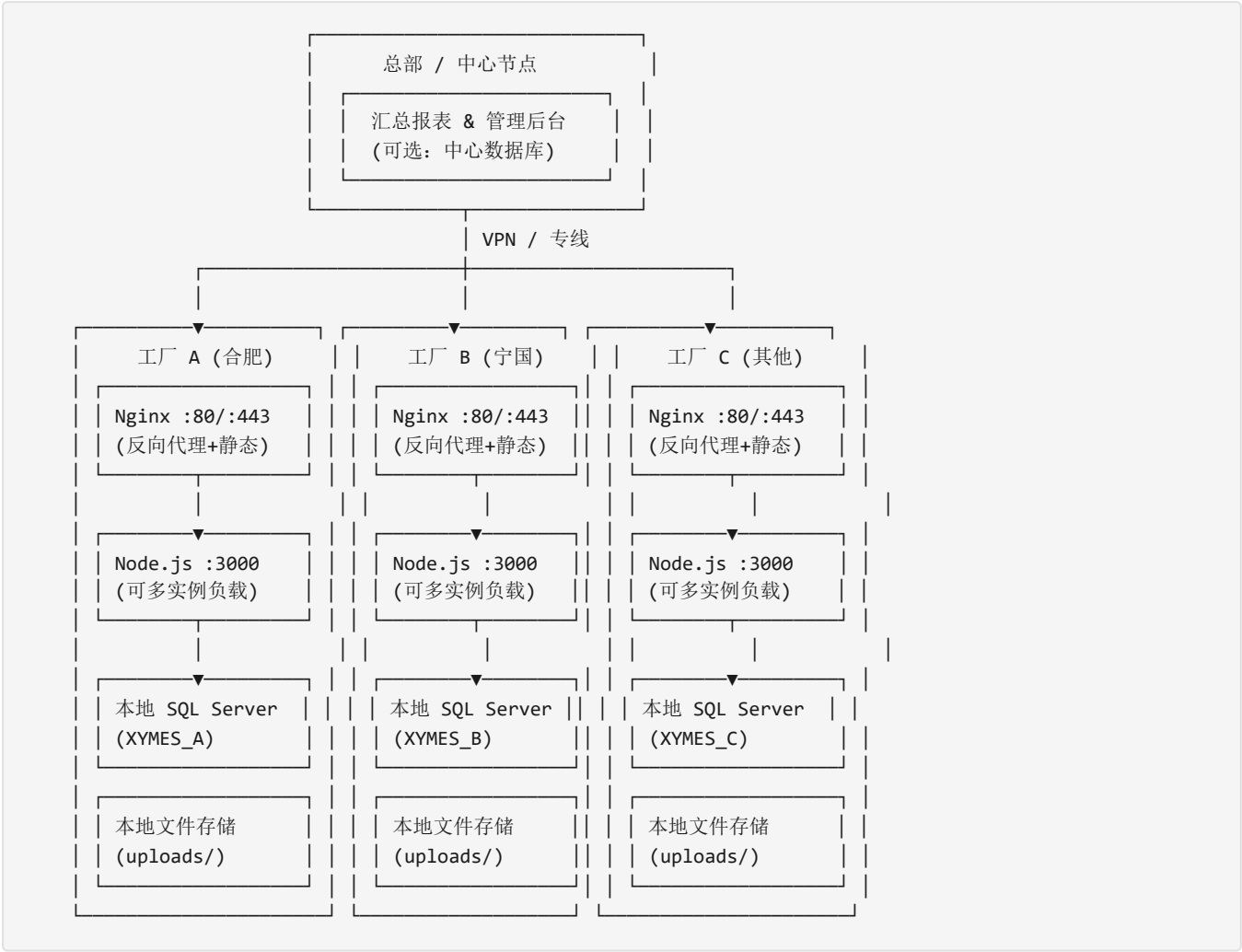
组件	现状	分布式瓶颈
后端	单进程 Express (server/src/server.ts), 端口3000	无状态 (JWT) , 天然可横向扩展
数据库	单 SQL Server 实例 (server/src/config/database.ts)	单点故障, 跨地域高延迟
文件存储	本地 uploads/ 目录 (Multer)	多节点间文件不共享
前端	Vite 构建产物, 后端托管静态文件	可独立部署到 CDN/各地 Nginx
编号生成	MAX() 查询自增 (如 M20260525001)	多节点并发可能冲突
Session	JWT (无状态)	天然支持分布式
定时任务	node-cron (server/src/services/)	多节点重复执行

1.2 已有有利条件

- JWT 认证无状态, 无需 Session 共享 (server/src/middleware/auth.middleware.ts)
- CORS 已支持多 Origin (server/src/app.ts L22-28)
- 已有 PM2 ecosystem.config.js 进程管理经验
- API 路由前缀统一 `/api/v1`
- 前端通过 Axios baseURL + Vite proxy 解耦

二、总体架构设计

2.1 目标拓扑



三、方案 A：中心数据库 + 多应用节点（统一数据源）

3.1 适用场景

- 工厂在同一省/市内，有可靠专线或 VPN（延迟 <10ms）
- 所有工厂需实时共享全部数据（如库存、订单状态）
- 管理层要求单一数据视图，不允许数据不一致

3.2 数据库层

架构：单一 SQL Server 实例部署在中心机房，所有工厂的应用节点直连该实例。

需改造内容：

文件	改造项	说明
server/.env	DB_HOST 指向中心数据库IP	`DB_HOST=192.168.10.1` (中心库地址)
server/src/config/database.ts	pool.max 上调至 30-50	多节点共享连接池，需放大上限
server/src/config/database.ts	增加 dialectOptions.requestTimeout	跨网络建议 60s+

风险缓解：

- 数据库主从复制（Always On 可用性组）：中心主库 + 异地只读副本，读操作走本地副本
- 应用层读写分离：报表查询走只读副本，写操作走主库

3.3 应用层

每工厂独立部署 Node.js 后端（server/），配置指向中心数据库。

需改造内容：

文件	改造项	说明
server/src/server.ts	增加实例标识环境变量	`INSTANCE_ID=factory_a`
server/src/config/database.ts	增加只读副本配置	`DB_READ_HOST` 环境变量
server/src/models/index.ts	编号生成前缀隔离	各厂使用不同前缀避免并发冲突
ecosystem.config.js	instances 改为多实例	`instances: 'max'`, `exec_mode: 'cluster'`

3.4 编号生成策略

多节点同时 INSERT 时，MAX() 查询方式会在高并发下产生重复编号。需改造：

改造文件：所有 controller 中的编号生成逻辑（约25+个 controller）

方案：使用 SQL Server SEQUENCE 对象或站点前缀：

```
-- 方案1: 数据库 SEQUENCE（推荐）
CREATE SEQUENCE seq_production_plan START WITH 1 INCREMENT BY 1;

-- 方案2: 站点前缀隔离（简单但编号不连续）
-- M20260525A001, M20260525B001...
```

3.5 文件存储

多节点需共享文件访问。

方案：

- 轻量级：NFS/SMB 共享目录，所有节点挂载同一 uploads/
- 专业级：MinIO 对象存储集群（S3 兼容）
- 云方案：阿里云 OSS / 腾讯云 COS

3.6 前端部署

每工厂部署独立 Nginx + 前端静态文件，或统一 CDN：

```
# 各工厂 Nginx 配置 (server/nginx/site.conf)
server {
    listen 80;
    server_name mes-factory-a.local;

    location / {
        root /opt/seals-mes/client/dist;
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header X-Instance-ID "factory_a";
    }
}
```

3.7 定时任务防重

`node-cron` 多节点会重复执行（如库存盘点调度 `server/src/services/stockCountScheduler.service.ts`）。

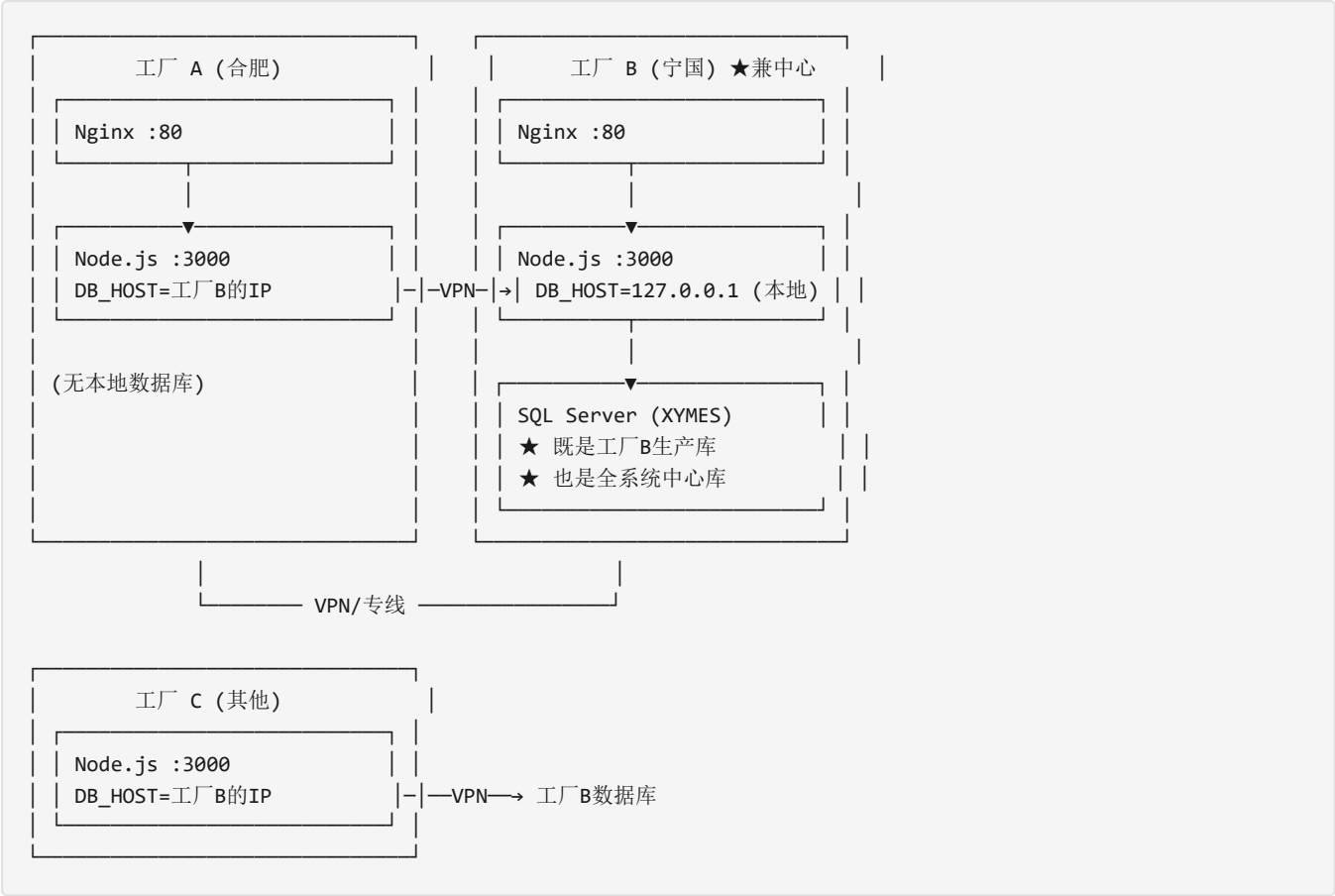
方案：增加分布式锁（Redis SETNX 或数据库锁）：

```
// 伪代码：仅持有锁的节点执行定时任务
const lock = await redis.set('cron:stock_count', instanceId, 'NX', 'EX', 60);
if (lock) { /* 执行 */ } else { /* 跳过 */ }
```

四、方案 A+：工厂数据库兼作中心库（混合模式）

4.1 核心思路

不用新建独立的中心数据库服务器，直接将工厂 B（或任一工厂）的数据库作为中心库，其他工厂的应用节点远程连接该库。这本质上是方案 A 的变体，只是“中心机房”恰好就是工厂 B 的机房。



4.2 适用场景

- 只有 2~3 个工厂，不想额外购置中心数据库服务器
- 工厂 B 的网络和服务器条件最好（已有高性能服务器 + 稳定公网/专线）
- 工厂 B 本身就是管理中枢（如总部所在地）

4.3 优势

优势	说明
零额外硬件成本	复用工厂 B 现有 SQL Server，无需新购服务器和数据库授权
部署最简单	工厂 B 完全不用改，其他厂只需改 DB_HOST 指向 B 的 IP
工厂 B 本地零延迟	B 厂应用直连 localhost，性能最佳
数据天然统一	所有厂写入同一库，查询报表无需汇总

4.4 风险与缓解

风险	影响	缓解措施
工厂 B 数据库宕机	全系统瘫痪（B厂本地也停）	Always On 可用性组 → 工厂 A 部署异步只读副本自动接管
工厂 B 服务器过载	B 厂本地操作变慢	连接池隔离：B 厂应用独占 10 连接，远程厂共享 20 连接
跨厂网络中断	工厂 A/C 无法使用系统	工厂 A/C 部署本地 SQL Server Express（免费版）作缓存应急 → 网络恢复后同步
远程写入延迟	A/C 厂用户体验下降	前端乐观更新 + 后端异步写入队列
单点磁盘故障	数据丢失	工厂 B 数据库定时备份 → 异地 (A厂NAS) 存储

4.5 关键配置要点

工厂 B (兼中心) — 无需大改：

```
# server/.env (工厂B)
DB_HOST=127.0.0.1          # 本地连接，零延迟
DB_PORT=1433
SITE_PREFIX=B              # 编号前缀
DEPLOY_MODE=hub            # 标识为中心节点
```

工厂 A (远程) — 仅改 DB_HOST：

```
# server/.env (工厂A)
DB_HOST=192.168.10.2       # 工厂B的IP
DB_PORT=1433
SITE_PREFIX=A              # 编号前缀，避免与B厂冲突
DEPLOY_MODE=remote         # 标识为远程节点
```

工厂 B 数据库连接池隔离：

```
// server/src/config/database.ts 改造
const isHub = process.env.DEPLOY_MODE === 'hub';

const sequelize = new Sequelize({
  // ...
  pool: {
    max: isHub ? 30 : 10,      // 中心节点承载更多连接
    min: isHub ? 5 : 2,
    acquire: 60000,           // 远程节点放宽获取超时
    idle: 60000
  }
});
```

4.6 灾备与高可用

正常状态：
工厂A/B/C → 工厂B SQL Server（主库）

故障切换（B库宕机）：
工厂B Always On 自动故障转移
→ 工厂A 的异步副本提升为主库
→ 所有节点自动重连新主库
(需 SQL Server Enterprise + Always On 可用性组)

网络中断应急（仅A厂断网）：
工厂A 自动切换 → 本地 SQL Server Express（应急缓存）
→ 仅支持生产报工、入库等核心操作
→ 网络恢复后自动同步回中心库

4.7 与纯方案 A 的差异

对比点	纯方案 A (独立中心库)	方案 A+ (B厂兼中心)
中心库服务器	独立购置	复用工厂B已有
B厂数据库负载	无（B厂也连独立中心库）	双重负载（本地+远程）
单点风险	中心库独立故障影响全部	B厂故障同时影响本地+远程
灾备策略	中心库独立灾备	需同时考虑B厂本地+中心两个角色
适用规模	3~5厂	2~3厂

五、方案 B：独立数据库 + 数据同步汇总（分布式自治）

5.1 适用场景

- 工厂跨省/市，网络不稳定或延迟高
- 各厂需独立运行（断网不影响生产）
- 总部仅需汇总报表和关键指标，不需实时操作数据

5.2 数据库策略

每工厂独立 SQL Server 实例，数据库名加后缀区分（XYMES_A, XYMES_B...）。

数据分类与同步策略：

数据类型	同步方式	同步频率	方向
基础数据（物料、客户、供应商等）	总部统一维护 → 下发	变更时实时/准实时	总部→工厂
生产数据（计划、报工、入库）	各厂独立，按需上报汇总	每小时/每天	工厂→总部
库存数据	各厂独立管理	实时汇总到总部视图	工厂→总部
销售订单	总部录入 → 下发工厂	实时	总部→工厂
用户/权限	总部统一维护 → 下发	变更时实时	总部→工厂
质量数据	各厂独立，定期上报	每天	工厂→总部

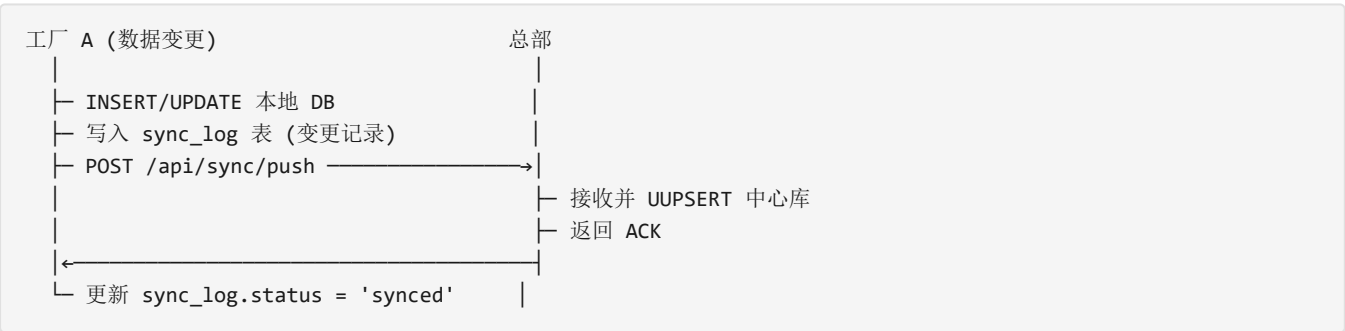
5.3 数据同步方案

方案 B-1：API 层同步（轻量级，推荐初期使用）

在现有 Express 基础上增加同步 API：

```
server/src/
├── modules/
│   └── sync/                                # 新增同步模块
│       ├── sync.controller.ts             # 数据收发控制器
│       ├── sync.routes.ts                 # /api/sync/push, /api/sync/pull
│       └── sync.service.ts                 # 同步逻辑（变更检测+冲突处理）
```

同步流程：



sync_log 表结构：

```
CREATE TABLE sync_log (
  id BIGINT IDENTITY PRIMARY KEY,
  table_name NVARCHAR(100),      -- 表名
  record_id NVARCHAR(100),       -- 记录主键
  operation CHAR(1),              -- C/U/D
  payload NVARCHAR(MAX),          -- 变更数据 JSON
  source_site NVARCHAR(20),       -- 来源站点
  status NVARCHAR(20) DEFAULT 'pending', -- pending/synced/failed
  created_at DATETIME DEFAULT GETDATE()
);
```

方案 B-2：消息队列同步（适合大数据量）

引入轻量级消息中间件（Redis Streams 或 RabbitMQ）：

工厂 A → Redis Streams → 总部消费者 → 中心库
工厂 B → 队列 →

方案 B-3: CDC + ETL 同步 (适合成熟期)

使用 SQL Server Change Tracking + 定时 ETL 任务:

- 各厂启用 `CHANGE_TRACKING = ON`
- 总部定时拉取增量变更
- 适用于对实时性要求不高的汇总场景

5.4 基础数据下发

总部维护主数据 → 推送到各工厂:

```
server/src/
├── modules/
│   └── master-data-sync/           # 新增主数据同步模块
│       ├── masterSync.controller.ts # 基础数据下发 API
│       ├── masterSync.service.ts    # 版本对比 + 增量更新
│       └── masterSync.validator.ts  # 数据校验
```

版本控制: 每个基础数据表增加 `version` 和 `last_synced_at` 字段, 仅同步增量变更。

5.5 编号生成

各工厂独立编号, 通过站点前缀区分:

站点	前缀示例	说明
合肥工厂 (HF)	M-HF-20260515001	生产计划
宁国工厂 (NG)	M-NG-20260515001	生产计划

改造点: `server/.env` 增加 `SITE_PREFIX=HF`, 所有编号生成函数读取该变量。

5.6 文件存储

各工厂独立存储, 总部汇总报表时通过 API 拉取关键文件 (如检验报告、入库单据扫描件)。

5.7 汇总报表

总部部署独立的报表服务, 从中心汇总库读取数据:

```
server/src/
├── modules/
│   └── report-aggregation/       # 新增汇总报表模块
│       ├── aggregate.controller.ts # 跨厂汇总查询
│       └── aggregate.service.ts    # UNION ALL 多库视图
```

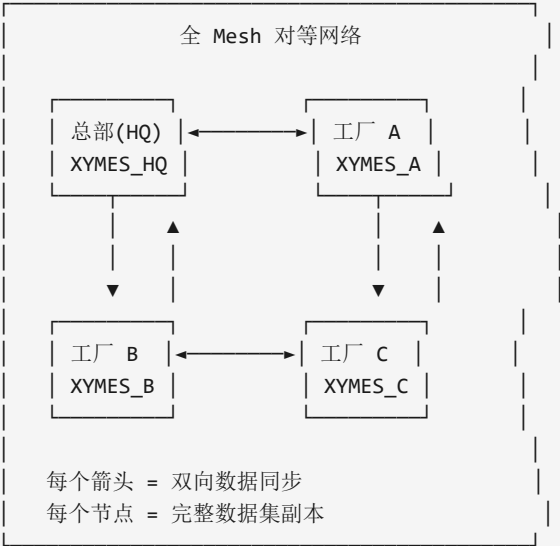
可使用 SQL Server Linked Server 或联邦查询实现跨库聚合:

```
-- 通过 Linked Server 查询
SELECT 'HF' AS site, * FROM [FACTORY_A].XYMES.dbo.production_order
UNION ALL
SELECT 'NG' AS site, * FROM [FACTORY_B].XYMES.dbo.production_order;
```

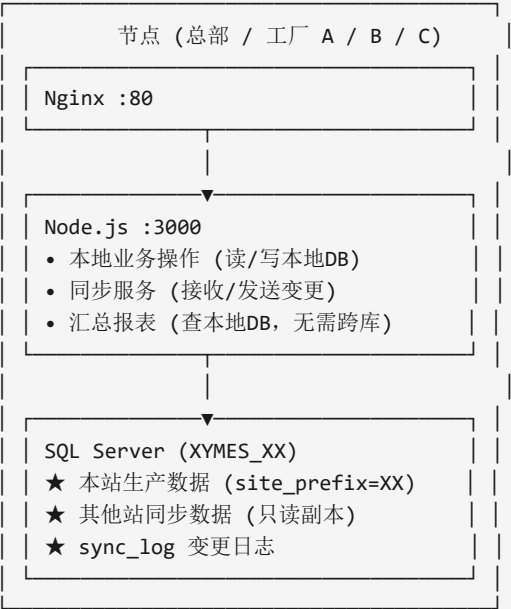
六、方案 B 变体：对等同步（总部无中心库）

6.1 核心理念

总部不设独立中心数据库，总部和所有工厂地位完全对等。每个节点（含总部）都拥有完整的本地 SQL Server 实例，节点之间**相互同步**全部业务数据。总部只是网络中的一个“普通节点”，通过自身本地库即可查看全系统汇总报表。



单个节点内部结构（所有节点完全一致）：



6.2 与原方案 B 的关键区别

维度	原方案 B (星型)	方案 B 变体 (对等 Mesh)
中心节点	总部有独立中心库	**无中心节点，全员对等**
数据流向	工厂→总部单向上报	**所有节点互相同步**
总部角色	数据汇总者+主数据下发者	**普通节点，与其他厂相同**
总部断网	无法汇总，但各厂正常	总部离线后恢复时补同步
主数据维护	总部独占修改	**任一节点可改，变更广播**
每个节点数据量	总部全量，工厂仅自己	**每个节点全量**
网络连接数	N 条 (工厂→总部)	$N \times (N-1) / 2$ 条 (全网状)

6.3 数据归属与冲突避免

核心原则：每条记录有且仅有一个"归属站点"，只有归属站点可以写入。其他站点以只读副本存储。

```
-- 所有业务主表增加 site_prefix 字段
ALTER TABLE production_order ADD site_prefix NVARCHAR(10) NOT NULL DEFAULT 'HQ';
ALTER TABLE work_report ADD site_prefix NVARCHAR(10) NOT NULL DEFAULT 'HQ';
ALTER TABLE stock_in ADD site_prefix NVARCHAR(10) NOT NULL DEFAULT 'HQ';
-- ... 所有业务表

-- 每个站点的应用层强制：写入时必须 site_prefix = 本机 SITE_PREFIX
-- 其他站点的数据只接受同步写入，不允许本地业务直接修改
```

同步时的数据隔离：

```
总部用户查看报表：
SELECT * FROM production_order WHERE site_prefix = 'HQ' -- 总部的数据
UNION ALL
SELECT * FROM production_order WHERE site_prefix = 'A' -- 工厂A同步来的数据
UNION ALL
SELECT * FROM production_order WHERE site_prefix = 'B' -- 工厂B同步来的数据
→ 全系统完整视图，但查询走本地库，零跨库延迟

工厂A用户查看报表：
同一条 SQL，同样能看到全系统数据（因为工厂A也同步了 HQ/B/C 的数据）
→ 每个节点都有完整副本
```

6.4 同步拓扑

轻量级方案 (3~4节点)：全 Mesh 直连同步

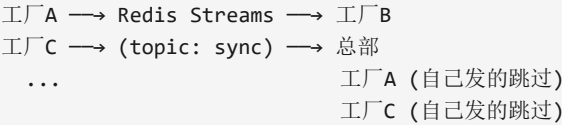
每个节点维护一个 `sync_peers` 配置：

```
# 总部 .env
SYNC_PEERS=http://factory-a:3000,http://factory-b:3000,http://factory-c:3000

# 工厂 A .env
SYNC_PEERS=http://hq:3000,http://factory-b:3000,http://factory-c:3000
```

- 同步流程（任一节点产生变更）：
- 1. 写入本地 DB + `sync_log`
 - 2. 遍历 `SYNC_PEERS`，POST `/api/sync/push` 到每个 `peer`
 - 3. 失败的 `peer` 标记 `retry`，后续补偿
 - 4. 每个 `peer` 接收后写入本地 DB（`site_prefix` 过滤，非本属数据只读）

扩展方案（5+节点）：引入轻量消息中间件解耦



优势：发送者不关心谁在线，消费者自行订阅

6.5 冲突处理策略

由于每条记录只有一个归属站点，**写入冲突从根本上被消除**。唯一需要处理的冲突场景：

冲突场景	处理策略
两站同时修改同一基础数据（如客户信息）	**乐观锁 + 版本号** ：后到达的变更发现版本号过期则拒绝，提示用户刷新后重试
网络分区恢复后，同一记录被两地修改	**Last-Write-Wins (LWW)** + 人工仲裁：以时间戳为准自动合并，记录到 <code>conflict_log</code> 供人工复核
编号重复（概率极低，因有站点前缀）	站点前缀天然隔离，不冲突

```
-- 冲突日志表
CREATE TABLE sync_conflict_log (
  id BIGINT IDENTITY PRIMARY KEY,
  table_name NVARCHAR(100),
  record_id NVARCHAR(100),
  local_version INT,
  remote_version INT,
  local_data NVARCHAR(MAX),
  remote_data NVARCHAR(MAX),
  resolution NVARCHAR(20) DEFAULT 'pending', -- pending/auto_resolved/manual
  created_at DATETIME DEFAULT GETDATE()
);
```

6.6 与方案 A/A+ 的角色类比

这个对等同步模型的精妙之处在于：**总部用户和工厂用户看到的界面、功能、报表完全一致**。

方案 A/A+: 所有节点看同一库 → 天然一致
方案 B 对等变体: 所有节点各有完整副本 → 同步后一致

对总部用户而言:

- 登录 `http://hq:80` → 看到全系统数据 (从本地HQ库查询)
- 查看"生产计划"列表 → 显示 HQ/A/B/C 所有工厂的计划
- 查看"汇总表" → 本地 SQL 计算, 无需跨库

对工厂A用户而言:

- 登录 `http://factory-a:80` → 同样看到全系统数据
- 但只能编辑 `site_prefix='A'` 的记录
- 其他站点的记录以只读形式展示

6.7 实施要点

要点	说明
每个节点全量存储	磁盘空间 = 单厂数据 × 节点数。橡胶密封件行业数据量小, 4节点×50GB=200GB, 普通服务器完全可承载
网络带宽需求低	仅同步增量变更, 非全量。日增量通常在 MB 级别
节点数上限	全Mesh模式建议≤5节点。超过5个建议引入消息队列解耦
新增节点	新节点启动时从任一已有节点拉取全量快照 → 后续增量同步
节点离线恢复	重新上线后从 <code>sync_log</code> 中拉取离线期间的增量 (基于 LSN/时间戳)
定时任务	库存盘点等全局任务, 只需一个节点执行 (分布式锁), 其他节点通过同步获得结果

七、公共基础改造 (三方案均需)

7.1 配置中心化

将环境变量升级为结构化配置:

```
server/src/config/  
├─ database.ts      → 不变  
├─ app.config.ts    → 新增: 站点标识、同步配置、日志级别  
├─ sync.config.ts   → 新增: 中心节点地址、同步间隔、重试策略  
└─ site.config.ts   → 新增: SITE_NAME, SITE_PREFIX, DEPLOY_MODE
```

7.2 前端 API 地址动态化

改造文件: `client/src/api/request.ts`, `client-mobile/src/api/request.ts`

```
// 从环境变量或配置文件读取 API 地址  
const API_BASE = import.meta.env.VITE_API_BASE || '/api/v1';
```

各厂构建时指定不同 API 地址:

```
# 工厂 A 构建  
VITE_API_BASE=https://mes-a.company.com/api/v1 npm run build
```

7.3 健康检查端点

新增文件: server/src/controllers/health.controller.ts

```
// GET /api/health
{
  "status": "ok",
  "site": "factory_a",
  "db": "connected",
  "uptime": 123456,
  "sync_status": "normal" // 方案B
}
```

7.4 日志集中化

使用 Winston 替代 Morgan + console.log, 输出 JSON 格式日志:

```
// server/src/utils/logger.util.ts (新增)
import winston from 'winston';

export const logger = winston.createLogger({
  format: winston.format.json(),
  defaultMeta: { site: process.env.SITE_PREFIX },
  transports: [
    new winston.transports.File({ filename: 'logs/app.log' }),
    // 可扩展: 发送到集中式日志服务 (Loki/ES)
  ]
});
```

7.5 部署自动化

新增 Ansible/Docker Compose 编排:

```
deploy/                                     # 新增目录
├── ansible/
│   ├── inventory/
│   │   ├── production.yml                 # 生产环境主机清单
│   │   └── staging.yml                   # 测试环境
│   ├── playbooks/
│   │   ├── deploy-backend.yml           # 后端部署剧本
│   │   ├── deploy-frontend.yml         # 前端部署剧本
│   │   └── setup-database.yml           # 数据库初始化
│   └── roles/
├── docker/
│   ├── Dockerfile.backend               # 后端容器化
│   ├── Dockerfile.frontend             # 前端容器化
│   └── docker-compose.per-site.yml      # 单站点编排
└── scripts/
    ├── init-site.sh                    # 新站点初始化脚本
    └── check-sync-status.sh             # 同步状态检查 (方案B)
```

八、四种方案对比

维度	方案 A：独立中心库	方案 A+：B厂兼中心	方案 B：星型同步	方案 B 变体：对等同步
数据一致性	强一致（实时）	强一致（实时）	最终一致（有延迟）	最终一致（有延迟）
断网可用性	低（工厂无法独立运行）	低（但B厂本地不受影响）	高（各厂独立运行）	**最高（全节点独立+互备）**
跨厂查询	天然支持	天然支持	需汇总库/Linked Server	**查本地库，零延迟**
实施复杂度	中（需新购服务器）	低（仅改DB_HOST）	高（需开发同步层）	高（需开发同步层+冲突处理）
运维复杂度	中（单库独立管理）	中低（B厂运维压力大）	中（多库+同步监控）	中（多库+全Mesh监控）
数据库压力	单点集中，独立承担	B厂双重负载	HQ承担汇总+下发	**每个节点均匀分担**
单点故障影响	仅中心库故障影响全部	B库故障 → 全系统+B厂本地	HQ故障不影响工厂	**无单点，任一点故障不影响其他**
总部是否需独立库	是	否（复用B厂）	是（有中心汇总库）	**否（HQ就是普通节点）**
硬件成本	高（独立服务器+授权）	低（复用B厂）	中（多套SQL Server）	中（每节点一套SQL Server）
编号冲突	需改造（SEQUENCE/前缀）	需改造（站点前缀）	站点前缀天然隔离	站点前缀天然隔离
主数据维护	天然统一	天然统一	HQ独占→下发	**任一节点可改→广播**
适用工厂数	≤5 个	2~3 个	不限	≤5 个（全Mesh）
每节点存储量	仅自身数据	仅自身数据	工厂少量/HQ全量	**每节点全量（但总数据量小）**

九、推荐实施路线

Phase 1 (1-2周): 基础准备

- └─ 站点配置化改造 (.env → 结构化配置)
- └─ 编号生成站点前缀
- └─ 健康检查端点
- └─ 日志标准化

Phase 2 (2-4周): 方案 A/A+ 落地

- └─ 方案A: 部署独立中心数据库 (Always On 可用性组)
- └─ 方案A+: 工厂B数据库直接作为中心库 (仅改其他厂DB_HOST)
- └─ 各厂应用节点部署
- └─ 文件存储 NAS/MinIO 部署
- └─ Nginx 反向代理配置
- └─ 定时任务分布式锁

Phase 3 (4-8周): 方案B 升级 (如需)

- └─ 方案B星型: 各厂独立数据库 + 总部汇总库
- └─ 方案B对等: 所有节点独立库 + 全Mesh互相同步
- └─ sync_log 表 + 变更捕获
- └─ 数据同步服务开发 (push/pull 或 peer broadcast)
- └─ 冲突处理机制 (乐观锁/版本号)
- └─ 主数据下发机制 (星型) / 变更广播 (对等)
- └─ 汇总报表服务
- └─ 同步监控告警

Phase 4 (持续): 运维体系

- └─ 集中式日志 (ELK/Loki)
- └─ 指标监控 (Prometheus + Grafana)
- └─ 自动化部署 (Ansible/Docker)
- └─ 灾备演练

十、文件改造清单总览

文件路径	改造内容	优先级
server/.env	新增 SITE_PREFIX, DEPLOY_MODE, SYNC_CENTER_URL	P0
server/src/config/database.ts	增加只读副本、连接池放大	P0
server/src/config/app.config.ts	**新增**：站点配置管理	P0
server/src/config/sync.config.ts	**新增**：同步配置，含 SYNC_PEERS 对等节点列表	P1
server/src/server.ts	实例标识、启动时注册到中心	P0
server/src/controllers/health.controller.ts	**新增**：健康检查接口	P0
server/src/modules/sync/*	**新增**：数据同步模块（push/pull + peer broadcast）	P1
server/src/modules/master-data-sync/*	**新增**：主数据下发（方案B）	P1
server/src/modules/report-aggregation/*	**新增**：汇总报表（方案B）	P1
server/src/utils/logger.util.ts	**新增**：结构化日志	P0
~25个 controller 文件	编号生成逻辑前置化	P0
server/src/services/stockCountScheduler.service.ts	分布式锁防重	P1
server/src/services/eventWorker.service.ts	分布式锁防重	P1
client/src/api/request.ts	API 地址动态化	P0
client-mobile/src/api/request.ts	API 地址动态化	P0
ecosystem.config.js	多实例 cluster 模式	P0
deploy/ 目录	**新增**：自动化部署脚本	P2